

APLICACIONES WEB • UNIDAD II

Desarrollar aplicaciones web con PERL e IntelliJ IDEA

Características y consideraciones clave

PERL para la web: ¿qué considerar?

Fortalezas de PERL

- Lenguaje maduro y pragmático; pionero de la web dinámica (CGI, años 90).
- Muy potente en texto y expresiones regulares.
- Enorme ecosistema CPAN: un módulo para casi todo.
- Multiplataforma; ideal para scripting, automatización e integración.

Para web moderna, ten presente

- CGI clásico hoy es legado: útil para aprender el modelo, no para producción nueva.
- El estándar moderno es PSGI/Plack (interfaz app web ↔ servidor).
- Frameworks: Mojolicious, Dancer2, Catalyst para proyectos serios.
- Para algo mínimo: un servidor propio con IO::Socket::INET (sin Apache).

El entorno: Strawberry Perl + IntelliJ IDEA

Strawberry Perl (Windows)

- Trae el intérprete, el compilador (gcc) y muchos módulos listos.
- Incluye DBI; el resto se instala con cpan / cpanm.
- Ejemplo: el driver de PostgreSQL DBD::Pg NO viene por defecto.
- Verifica con perl -v en la terminal antes de empezar.

IntelliJ IDEA

- Plugin «Perl» (Camelcade): resaltado, autocompletado, ejecución y depuración.
- Hay que configurar el intérprete (perl.exe), no wperl.exe.
- Estructura del proyecto: lib (módulos) y t (pruebas).
- Integración con Git para control de versiones.

Qué considerar en el IDE

1

Intérprete

Configura el Perl5 Interpreter apuntando a perl.exe (C:\Strawberry\perl\bin).

2

Plugin Perl

Instala Camelcade desde el Marketplace o, si la red lo bloquea, desde disco (.zip).

3

Run & Debug

Ejecuta con el botón verde; usa breakpoints y la consola integrada para depurar.

4

Red institucional

Con inspección SSL (firewall UTEQ): acepta el certificado o instala plugins desde disco.

Cómo estructurar la aplicación web

Modelo y servidor

- Modelo request-response; HTTP es sin estado (usa cookies/sesiones para recordar).
- Formas de servir: CGI (Apache), servidor propio (IO::Socket::INET) o PSGI/Plack.
- Enrutamiento: decide qué hacer según el método (GET/POST) y la ruta.
- Genera la respuesta con el código de estado y cabeceras correctas.

Interacción con el usuario

- GET para consultar (idempotente); POST para enviar o modificar datos.
- Decodifica los parámetros del formulario (URL-decode).
- Patrón Post-Redirect-Get: redirige tras un POST para evitar reenvíos.
- Separa la lógica (Perl) de la presentación (plantillas o heredocs ordenados).

Acceso a la base de datos

DBI + driver

- DBI es la interfaz común para cualquier base de datos.
- El driver concreto: DBD::Pg para PostgreSQL (instálalo con `cpan DBD::Pg`).
- Conexión con DSN + atributos: `RaiseError`, `AutoCommit`, `pg_enable_utf8`.
- Cierra siempre lo que abres (`finish`, `disconnect`).

Buenas prácticas de datos

- Sentencias preparadas con «?»: seguras y reutilizables.
- Nunca concatenar datos del usuario dentro del SQL.
- Para proyectos grandes existe un ORM: `DBIx::Class`.
- Versiona el esquema (un archivo `schema.sql`).

Lo no negociable

Validar la entrada

En el SERVIDOR. El cliente es manipulable; nunca confíes solo en él.

Escapar la salida

Convierte < > & " en entidades para evitar XSS al mostrar datos.

Sentencias preparadas

Marcadores «?» en TODA consulta: defensa contra SQL Injection.

Credenciales

Claves con \$ o @ en comillas simples; mejor fuera del código (variables de entorno).

UTF-8 de extremo a extremo

Archivo, salida, base de datos y cabecera HTTP en UTF-8.

Diseño y mantenimiento del código

Disciplina de código

- use strict; y use warnings; SIEMPRE.
- Funciones pequeñas, cada una con una sola responsabilidad.
- Manejo de errores claro (RaiseError, or die con mensajes útiles).
- Comentarios que expliquen el porqué, no solo el qué.

Organización y entrega

- Separa lógica de presentación (plantillas: Template Toolkit).
- Reutiliza con subrutinas y módulos (.pm en lib/).
- Versiona con Git; nunca subas secretos al repositorio.
- Prueba tus scripts (carpeta t/) y verifica en otra máquina.

Checklist rápido

Entorno: intérprete perl.exe configurado · plugin Perl instalado · módulos vía cpan.

Arquitectura: request-response · enrutamiento por método y ruta · patrón PRG.

Datos: DBI + DBD::Pg · sentencias preparadas · cerrar la conexión.

Seguridad: validar entrada · escapar salida · credenciales protegidas · UTF-8.

Calidad: strict/warnings · separar lógica y presentación · Git sin secretos · pruebas.

Estas consideraciones son transversales: aplican igual al pasar a PHP, Java o ASP.NET en el resto de la Unidad II.